04. HTTP 트래커 구현 상세 분석

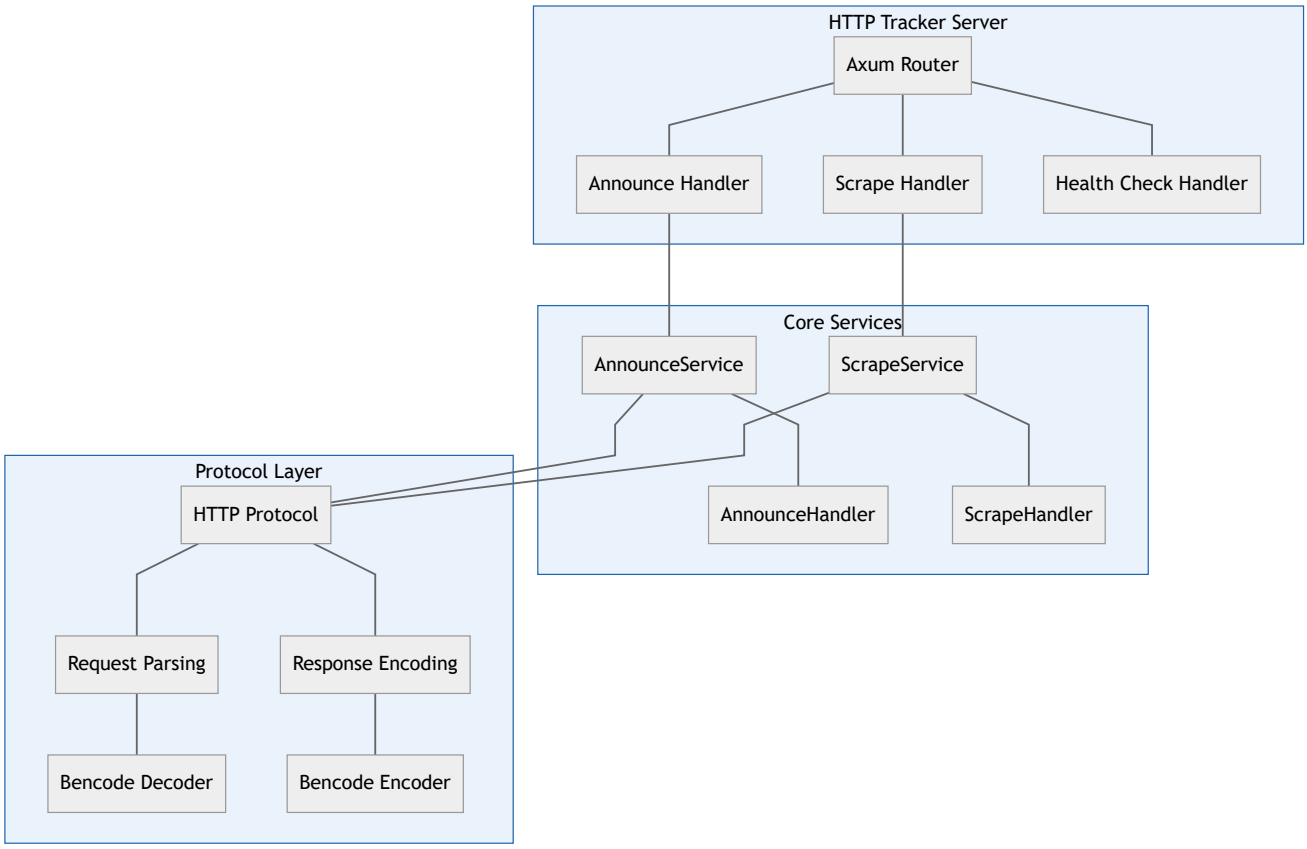
4.1 BEP 3 프로토콜 개요

Torrust Tracker의 HTTP 트래커는 [BEP 3: The BitTorrent Protocol Specification]을 기반으로 구현되었으며, 다음과 같은 추가 BEP들을 지원합니다:

- **BEP 7**: IPv6 Tracker Extension - IPv6 주소 지원
- **BEP 23**: Tracker Returns Compact Peer Lists - 압축된 피어 리스트 형식
- **BEP 48**: Tracker Protocol Extension: Scrape - 토렌트 통계 조회

4.1.1 HTTP 트래커 아키텍처

HTTP 트래커는 Axum 웹 프레임워크를 기반으로 구현되어 있으며, 다음과 같은 구조를 가집니다:



4.1.2 지원하는 엔드포인트

HTTP 트래커는 다음 엔드포인트들을 제공합니다:

- **`/announce`** : 피어 등록 및 스웜 정보 조회
- **`/announce/{key}`** : 인증이 필요한 피어 등록 (프라이빗 모드)
- **`/scrape`** : 토렌트 통계 정보 조회
- **`/scrape/{key}`** : 인증이 필요한 통계 조회 (프라이빗 모드)

- **`/health_check`** : 서버 상태 확인

4.2 Announce 엔드포인트 (`/announce`) 구현 분석

4.2.1 라우팅 설정

`packages/axum-http-tracker-server/src/v1/routes.rs` 에서 라우팅이 설정됩니다:

```
pub fn router(http_tracker_container: &Arc<HttpTrackerCoreContainer>, server_s
Router::new()
// Announce request
.route(
"/announce",
get(announce::handle_without_key).with_state((
http_tracker_container.announce_service.clone(),
server_service_binding.clone(),
)),
)
.route(
"/announce/{key}",
get(announce::handle_with_key).with_state((
http_tracker_container.announce_service.clone(),
server_service_binding.clone(),
)),
)
```

```
// 미들웨어 설정
.layer(SecureClientIpSource::ConnectInfo.into_extension())
.layer(CompressionLayer::new())
.layer(SetRequestIdLayer::x_request_id(MakeRequestUuid))
// ... 기타 미들웨어
}
```

4.2.2 Announce 핸들러 구현

`packages/axum-http-tracker-server/src/v1/handlers/announce.rs` 에서 요청을 처리합니다:

인증 없는 Announce 처리 (`handle_without_key()`)

```
pub  async  fn  handle_without_key(
State(state): State<(Arc<AnnounceService>, ServiceBinding)>,
ExtractRequest(announce_request): ExtractRequest,
ExtractClientIpSources(client_ip_sources): ExtractClientIpSources,
) -> Response {
tracing::debug!("http announce request: {:#?}", announce_request);

handle(&state.0, &announce_request, &client_ip_sources, &state.1, None).await
}
```

인증이 필요한 Announce 처리 (`handle_with_key()`)

```
pub  async  fn  handle_with_key(

State(state): State<(Arc<AnnounceService>, ServiceBinding)>,

ExtractRequest(announce_request): ExtractRequest,

ExtractClientIpSources(client_ip_sources): ExtractClientIpSources,

ExtractKey(key): ExtractKey,

) -> Response {

tracing::debug!("http announce request: {:#?}", announce_request);



handle(&state.0, &announce_request, &client_ip_sources, &state.1, Some(key)).awa

}
```

4.2.3 Announce 서비스 로직

`packages/http-tracker-core/src/services/announce.rs` 의 `AnnounceService` 에서 핵심 로직을 처리합니다:

```
pub  async  fn  handle_announce(

&self,

announce_request: &Announce,

client_ip_sources: &ClientIpSources,

server_service_binding: &ServiceBinding,

maybe_key: Option<Key>,

) -> Result<AnnounceData, HttpAnnounceError> {

// 1. 인증 처리
```

```
self.authenticate(maybe_key).await?;


// 2. 권한 확인 (화이트리스트)
self.authorize(announce_request.info_hash).await?;


// 3. 클라이언트 IP 주소 해석
let remote_client_addr = resolve_remote_client_addr(
&self.core_config.net.on_reverse_proxy.into(),
client_ip_sources
)?;


// 4. 피어 정보 생성
let mut peer = peer_from_request(announce_request, &remote_client_addr.ip());


// 5. 요청할 피어 수 결정
let peers_wanted = Self::peers_wanted(announce_request);


// 6. 코어 핸들러에 위임
let announce_data = self
.announce_handler
.handle_announcement(
&announce_request.info_hash,
&mut peer,
```

```
&remote_client_addr.ip(),
&peers_wanted,
)
.await?;

// 7. 통계 이벤트 발송
self.send_event(
announce_request.info_hash,
remote_client_addr,
server_service_binding.clone(),
peer,
).await;

Ok(announce_data)
}
```

4.3 Scrape 엔드포인트 (`/scrape`) 구현 분석

4.3.1 Scrape 핸들러 구현

`packages/axum-http-tracker-server/src/v1/handlers/scrape.rs` 에서 scrape 요청을 처리합니다:

```
pub  async  fn  handle_without_key(
```

```
State(state): State<(Arc<ScrapeService>, ServiceBinding)>,

ExtractRequest(scrape_request): ExtractRequest,

ExtractClientIpSources(client_ip_sources): ExtractClientIpSources,

) -> Response {

tracing::debug!("http scrape request: {:#?}", &scrape_request);



handle(&state.0, &scrape_request, &client_ip_sources, &state.1, None).await

}
```

4.3.2 Scrape 서비스 로직

`packages/http-tracker-core/src/services/scrape.rs` 의 `ScrapeService` 에서 처리합니다:

```
pub  async  fn  handle_scrape(

&self,

scrape_request: &Scrape,

client_ip_sources: &ClientIpSources,

server_service_binding: &ServiceBinding,

maybe_key: Option<Key>,

) -> Result<ScrapeData, HttpScrapeError> {

// 인증이 필요하고 인증되지 않은 경우 빈 데이터 반환

let  scrape_data = if  self.authentication_is_required() && !self.is_authenticat

ScrapeData::zeroed(&scrape_request.info_hashes)

} else {
```

```
self.scrape_handler.handle_scrape(&scrape_request.info_hashes).await?
};

// 클라이언트 IP 해석
let remote_client_addr = resolve_remote_client_addr(
&self.core_config.net.on_reverse_proxy.into(),
client_ip_sources
)?;

// 통계 이벤트 발송
self.send_event(remote_client_addr, server_service_binding.clone()).await;

Ok(scrape_data)
}
```

4.4 HTTP 요청 파라미터 검증 및 파싱

4.4.1 Announce 요청 파라미터

`packages/http-protocol/src/v1/requests/announce.rs` 에서 Announce 요청 구조를 정의합니다:

```
pub struct Announce {
// 필수 파라미터
```

```
pub info_hash: InfoHash, // 토렌트 식별자 (20바이트 SHA1)
pub peer_id: PeerId, // 피어 식별자 (20바이트)
pub port: u16, // 피어 포트

// 선택적 파라미터
pub downloaded: Option<NumberOfBytes>, // 다운로드한 바이트 수
pub uploaded: Option<NumberOfBytes>, // 업로드한 바이트 수
pub left: Option<NumberOfBytes>, // 남은 바이트 수
pub event: Option<Event>, // 이벤트 (started, stopped, completed)
pub compact: Option<Compact>, // 압축 모드 여부
pub numwant: Option<u32>, // 요청할 피어 수
}
```

4.4.2 파라미터 추출 및 검증

각 파라미터는 전용 함수로 추출되고 검증됩니다:

```
// Info Hash 추출 (퍼센트 인코딩된 20바이트)
fn extract_info_hash(query: &Query) -> Result<InfoHash, ParseAnnounceQueryError
match query.get_param(INFO_HASH) {
Some(raw_param) => {
Ok(percent_decode_info_hash(&raw_param)
.map_err(|err| ParseAnnounceQueryError::InvalidInfoHashParam {
param_name: INFO_HASH.to_owned(),
```

```
param_value: raw_param.clone(),
source: Located(err).into(),
})?)
}
None => Err(ParseAnnounceQueryError::MissingParam {
location: Location::caller(),
param_name: INFO_HASH.to_owned(),
}),
}
}

// Peer ID 추출 (퍼센트 인코딩된 20바이트)
fn  extract_peer_id(query: &Query) -> Result<PeerId, ParseAnnounceQueryError> {
match  query.get_param(PEER_ID) {
Some(raw_param) => Ok(percent_decode_peer_id(&raw_param)
.map_err(|err| ParseAnnounceQueryError::InvalidPeerIdParam {
param_name: PEER_ID.to_owned(),
param_value: raw_param.clone(),
source: Located(err).into(),
})?),
None => Err(ParseAnnounceQueryError::MissingParam {
location: Location::caller(),
param_name: PEER_ID.to_owned(),
```

```
}),
}
}
```

4.4.3 이벤트 타입 처리

```
#[derive(PartialEq, Debug, Clone)]
pub enum Event {
Started, // 다운로드 시작
Stopped, // 다운로드 중단
Completed, // 다운로드 완료
Empty, // 정기 업데이트
}

impl FromStr for Event {
type Err = ParseAnnounceQueryError;

fn from_str(raw_param: &str) -> Result<Self, Self::Err> {
match raw_param {
"started" => Ok(Self::Started),
"stopped" => Ok(Self::Stopped),
"completed" => Ok(Self::Completed),
"empty" => Ok(Self::Empty),
```

```
_ => Err(ParseAnnounceQueryError::InvalidParam {
param_name: EVENT.to_owned(),
param_value: raw_param.to_owned(),
location: Location::caller(),
}),
}
}
}
```

4.5 Bencode 형식의 응답 생성 과정

4.5.1 응답 데이터 구조

HTTP 트래커는 Bencode 형식으로 응답을 생성합니다. `packages/http-protocol/src/v1/responses/announce.rs` 에서 정의됩니다:

```
pub struct Announce<E>
where
E: From<AnnounceData> + Into<Vec<u8>>,
{
pub data: E,
}
```

4.5.2 응답 빌드 과정

핸들러에서 응답을 빌드하는 과정:

```
fn build_response(announce_request: &Announce, announce_data: AnnounceData) ->
if announce_request.compact.as_ref().is_some_and(|f| *f == Compact::Accepted) {
// Compact 모드 응답
let response: responses::Announce<responses::Compact> = announce_data.into();
let bytes: Vec<u8> = response.data.into();
(StatusCode::OK, bytes).into_response()
} else {
// Normal (Non-Compact) 모드 응답
let response: responses::Announce<responses::Normal> = announce_data.into();
let bytes: Vec<u8> = response.data.into();
(StatusCode::OK, bytes).into_response()
}
}
```

4.6 Compact vs Non-Compact 피어 리스트 형식

4.6.1 Normal (Non-Compact) 형식

Normal 형식은 각 피어를 개별 딕셔너리로 표현합니다:

```
pub struct Normal {
complete: i64, // 완료한 피어 수
incomplete: i64, // 다운로드 중인 피어 수
```

```
interval: i64, // 다음 announce 간격 (초)
min_interval: i64, // 최소 announce 간격 (초)
peers: Vec<NormalPeer>, // 피어 리스트
}

pub struct NormalPeer {
pub peer_id: [u8; 20], // 피어 ID
pub ip: IpAddr, // IP 주소
pub port: u16, // 포트 번호
}
```

Bencode 인코딩 과정:

```
impl Into<Vec<u8>> for Normal {
fn into(self) -> Vec<u8> {
let mut peers_list = ben_list!();
let peers_list_mut = peers_list.list_mut().unwrap();
for peer in &self.peers {
peers_list_mut.push(peer.into());
}

(ben_map! {
"complete" => ben_int!(self.complete),
"incomplete" => ben_int!(self.incomplete),
```

```
"interval" => ben_int!(self.interval),
"min interval" => ben_int!(self.min_interval),
"peers" => peers_list.clone()
}).encode()
}
}
```

예시 응답 (JSON 표현):


```
{
"complete": 333,
"incomplete": 444,
"interval": 111,
"min interval": 222,
"peers": [
{
"ip": "105.105.105.105",
"peer id": "-qB00000000000000001",
"port": 28784
}
]
}
```

4.6.2 Compact 형식

Compact 형식은 피어 정보를 바이너리로 압축하여 전송합니다:

```
pub struct Compact {
complete: i64,
incomplete: i64,
interval: i64,
min_interval: i64,
peers: Vec<u8>, // IPv4 피어들 (6바이트씩)
peers6: Vec<u8>, // IPv6 피어들 (18바이트씩)
}
```

압축 인코딩 과정:

```
impl FromIterator<CompactPeerData<Ipv4Addr>> for CompactPeersEncoded {
fn from_iter<T: IntoIterator<Item = CompactPeerData<Ipv4Addr>>>(iter: T) -> Sel
let mut bytes: Vec<u8> = vec![];

for peer in iter {
// IPv4 주소 (4바이트) + 포트 (2바이트) = 6바이트
bytes.write_all(&u32::from(peer.ip).to_be_bytes())
.expect("it should write peer ip");
bytes.write_all(&peer.port.to_be_bytes())
.expect("it should write peer port");
```

```
}

bytes.into()
}
}
```

예시 응답 (JSON 표현):

```
{
"complete": 333,
"incomplete": 444,
"interval": 111,
"min interval": 222,
"peers": "iiiipp", // IPv4 피어들 (바이너리)
"peers6": "iiiiiiiiiiiiiiipp" // IPv6 피어들 (바이너리)
}
```

4.6.3 형식 선택 로직

클라이언트의 `compact` 파라미터에 따라 응답 형식이 결정됩니다:

```
#[derive(PartialEq, Debug)]
pub enum Compact {
Accepted = 1, // compact=1: Compact 형식 선호
NotAccepted = 0, // compact=0: Normal 형식 선호
```

```
}
```

- `compact=1` 또는 `compact` 파라미터가 있으면 Compact 형식
- `compact=0` 또는 파라미터가 없으면 Normal 형식 (기본값)

4.6.4 장단점 비교

| 특성 | Normal 형식 | Compact 형식 |

|------|----------|----------|

| **크기** | 큼 (각 피어당 ~60바이트) | 작음 (IPv4: 6바이트, IPv6: 18바이트) |

| **피어 ID** | 포함 | 미포함 |

| **가독성** | 높음 (텍스트 기반) | 낮음 (바이너리) |

| **파싱 복잡도** | 낮음 | 높음 |

| **네트워크 효율성** | 낮음 | 높음 |

| **메모리 사용량** | 높음 | 낮음 |

4.7 에러 처리 및 응답

4.7.1 에러 응답 형식

모든 에러는 Bencode 형식의 에러 응답으로 반환됩니다:

```
pub struct Error {
pub failure_reason: String,
}
```

```
impl Error {
pub fn write(&self) -> Vec<u8> {
(ben_map! {
"failure reason" => ben_bytes!(self.failure_reason.clone())
}).encode()
}
}
```

4.7.2 주요 에러 시나리오

1. **인증 실패 (프라이빗 모드)**

```
"failure_reason": "Tracker core error: Tracker core authentication error: Missin
```

2. **화이트리스트 위반 (Listed 모드)**

```
"failure_reason": "Tracker core error: Tracker core whitelist error: The torrent
```

3. **IP 해석 실패 (리버스 프록시 환경)**

```
"failure_reason": "Error resolving peer IP: missing or invalid the right most X-
```

4. **파라미터 파싱 에러**

```
"failure_reason": "Bad request. Cannot parse query params for announce request:
```

4.8 성능 최적화 및 미들웨어

4.8.1 적용된 미들웨어

HTTP 트래커는 다음 미들웨어들을 사용하여 성능과 관찰성을 향상시킵니다:

```
.layer(CompressionLayer::new()) // 응답 압축
.layer(SetRequestIdLayer::x_request_id(MakeRequestUuid)) // 요청 ID 생성
.layer(PropagateHeaderLayer::new(HeaderName::from_static("x-request-id"))) // 요
.layer(TraceLayer::new_for_http() // HTTP 요청/응답 로깅
.make_span_with(DefaultMakeSpan::new().level(Level::INFO))
.on_request(|request, span| { /* 요청 로깅 */ })
.on_response(|response, latency, span| { /* 응답 로깅 */ }))
.layer(TimeoutLayer::new(DEFAULT_TIMEOUT)) // 요청 타임아웃
```

4.8.2 클라이언트 IP 해석

리버스 프록시 환경을 고려한 클라이언트 IP 해석:

```
.layer(SecureClientIpSource::ConnectInfo.into_extension())
```

이를 통해 `X-Forwarded-For` 헤더나 연결 정보에서 실제 클라이언트 IP를 추출합니다.

4.8.3 통계 및 모니터링

각 요청은 통계 이벤트를 발생시켜 모니터링이 가능합니다:

```
// Announce 이벤트

let  event = Event::TcpAnnounce {

connection: event::ConnectionContext::new(remote_client_addr, server_service_bin

info_hash,

announcement,

};



// Scrape 이벤트

let  event = Event::TcpScrape {

connection: ConnectionContext::new(remote_client_addr, server_service_binding),

};
```

이러한 구조를 통해 Torrust Tracker의 HTTP 구현은 BEP 3 표준을 완전히 준수하면서도 현대적인 웹 프레임워크의 장점을 활용하여 높은 성능과 확장성을 제공합니다.